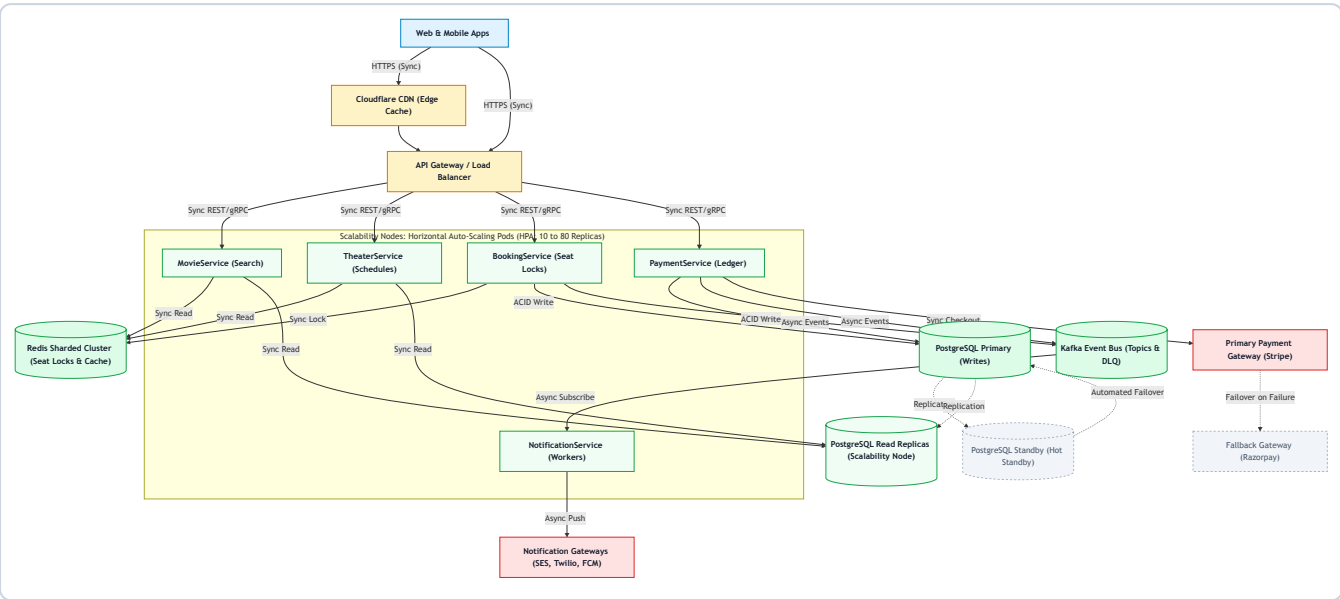


1. End-to-End Architecture Diagram (with Scalability Nodes & Failover Paths)



2. Communication Protocols: Synchronous vs. Asynchronous Workflows

Interaction Path	Pattern / Protocol	Target Latency	Architectural Rationale
Catalog Search & Schedule Browsing	Synchronous (HTTP/2 / REST)	p95 < 80ms	Direct read paths backed by CDN edges and Redis caches ensure instant user feedback when discovering movies.
Seat Selection & Lock Acquisition	Synchronous (gRPC / Internal)	p95 < 50ms	Requires atomic, immediate confirmation of temporary reservation to prevent competing users from holding identical seats.
Payment Verification & Webhooks	Synchronous (HTTPS Webhook)	p95 < 1500ms	Payment provider informs platform of authorization status synchronously; booking state updates immediately upon response.
Ticket Generation & Notification Fan-Out	Asynchronous (Kafka Topics)	Decoupled (1 - 3s)	Non-blocking post-booking tasks (QR generation, email, SMS, push) are offloaded to Kafka to guarantee sub-second user checkout.
Auditing, Analytics & Search Ingestion	Asynchronous (Event Streaming)	Decoupled (Near Real-Time)	Downstream analytics and business dashboards ingest event streams without introducing overhead to transactional databases.

3. Component Deep Descriptions

Web & Mobile Applications

Client apps providing responsive browsing, real-time seat selection via WebSockets, and seamless checkout flows across devices.

Partner Admin Portal

Administrative dashboard for theater partners to configure auditoriums, map seating layouts, and publish schedules.

Global CDN (Cloudflare)

Terminates TLS, absorbs static traffic, and caches read-heavy catalog search results close to users geographically.

API Gateway / Load Balancer

Central entry point managing SSL termination, JWT authentication, request routing, and distributed rate limiting.

MovieService (Scalable Node)

Stateless search pods scaled horizontally via Kubernetes HPA; manages global movie metadata and high-volume read filters.

TheaterService (Scalable Node)

Maintains venue directory, auditorium configurations, seating layouts, and daily screen showtime calendars.

BookingService (Scalable Node)

Orchestrates 10-minute temporary seat locks, validates checkout carts, and commits final confirmed reservations.

PaymentService (Scalable Node)

Coordinates with external payment providers, handles idempotent transaction webhooks, and manages ledger updates.

NotificationService (Scalable Node)

Asynchronously consumes booking events to dispatch receipts, tickets, and reminders via Email, SMS, and Push.

Redis Sharded Cluster

High-performance in-memory store managing distributed seat locks, cacheable show schedules, and active sessions.

PostgreSQL Primary & Read Replicas

ACID primary for transactional writes, replicating to hot standby for automated failover and read replicas for scale.

Kafka Event Streaming Cluster

High-throughput event bus handling async booking confirmations, notification queues, and dead-letter failure topics.

4. End-to-End Request Life Cycle

- **Stage 1 (Discovery & Browsing):** The client requests current showings for a city. The API Gateway routes the request to MovieService and TheaterService, which resolve data directly from CDN edge caches and Redis replicas within 80ms.
- **Stage 2 (Seat Selection & Hold):** The user selects desired seats on the interactive map. BookingService verifies availability and places a 10-minute temporary hold in the Redis distributed lock layer. If another user simultaneously attempts the same seats, the lock fails cleanly and prompts the user to select alternative seats.
- **Stage 3 (Payment Checkout):** With the hold active, the client submits payment details. PaymentService forwards the payload to an external gateway (Stripe/Razorpay). Upon successful gateway authorization, PaymentService triggers a database transaction in PostgreSQL to convert the temporary hold into a permanent booking.
- **Stage 4 (Async Confirmation & Fan-Out):** BookingService emits a BookingConfirmedEvent to Kafka. The client receives an immediate success response on screen. In parallel, NotificationService workers consume the Kafka event to generate the digital ticket with QR code and dispatch multi-channel confirmation messages.